

Leetcode 743. Network Delay Time

Achyutha Duddebanda

May 26th 2026

Problem

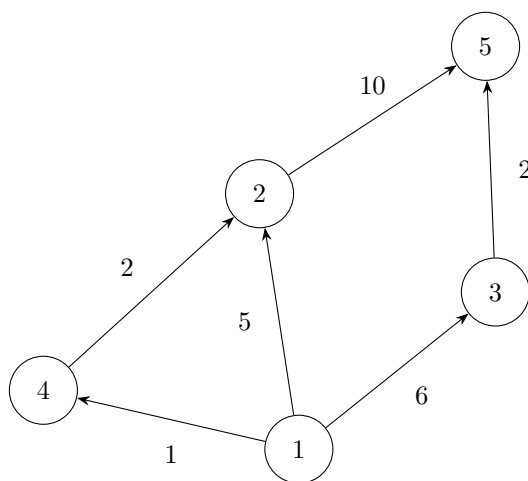
In this problem, we are given a network (or directed graph) of n nodes labeled from 1 to n . We are also given a list of travel times stored in the array `times`, which stores the travel times for each signal to its respective recipient (node) and a single source node k . The data in the `times` array is stored as the following:

$$\text{times}[i] = (u_i, v_i, w_i)$$

where u_i is the source node, v_i is the destination node, and w_i is the time taken for the signal to travel to the destination. Our goal is to return the minimum time taken for all nodes to receive the signal from the source node.

Example

Consider the following directed graph with the source vertex at node 1:



To find the minimum amount of time for all the nodes to receive the signal, we need to determine when each node receives the signal and figure out how long

it takes until the last node receives the signal. In this case, the last node that receives the signal is node 5 at time 8 ($6 + 2 = 8$). Therefore, we return 8 as our final answer.

Approach

In this problem, the signal will always take the shortest path from the source vertex to the destination vertex. Since there is only one given source node, k , we are dealing with the single-source shortest paths problem. Since all the edge weights are positive, we can apply **Dijkstra's algorithm** to this problem. Here are the steps applied to the graph in the previous section:

1. Create an adjacency list
 - Make sure that the weights are also stored here. Use a pair to store the weights for each edge for easy access.
2. Create a `dist` array that will store the shortest distances from the source to that vertex. Initialize the source to 0 and the rest as ∞ (meaning undiscovered nodes). For our example,

$$[0, \infty, \infty, \infty, \infty]$$

3. Initialize a priority queue that is sorted based on weights. This allows us to always process the least costly edge weight first. Add the source node with its corresponding weight of 0.
4. While the queue is not empty:
 - Get the current node and its neighbors.
 - For each neighbor:
 - Check if this node is faster to get to going through that neighbor.

$$\text{dist}[n - 1] > \text{dist}[s - 1] + d$$

- If it is, update the distance to the shortest path and add it back into the priority queue.
5. After applying this algorithm, `dist` will store the shortest path from the source to all the other nodes in the graph. For the example graph, this will be:

$$[0, 3, 6, 1, 8]$$

6. Get the maximum value from the `dist` array. This is the final output.

$$\boxed{8}$$

Time and Space Complexity

- Time complexity: $\mathcal{O}(E \log V)$
- Space complexity: $\mathcal{O}(V)$

where E, V are the numbers of edges and vertices, respectively. The additional $\log V$ time comes from the cost of polling an element from a priority queue. Without using a priority queue, the time complexity would be $\mathcal{O}(V^2)$ since all the vertices have to be scanned again to retrieve the shortest distance.

Tips

- In order to avoid re-searching for the weights for every iteration, store the edge weights directly in the adjacency list. In this case, I used a pair to do this.
- Use a priority queue to avoid additional iterations and to make the problem less confusing.